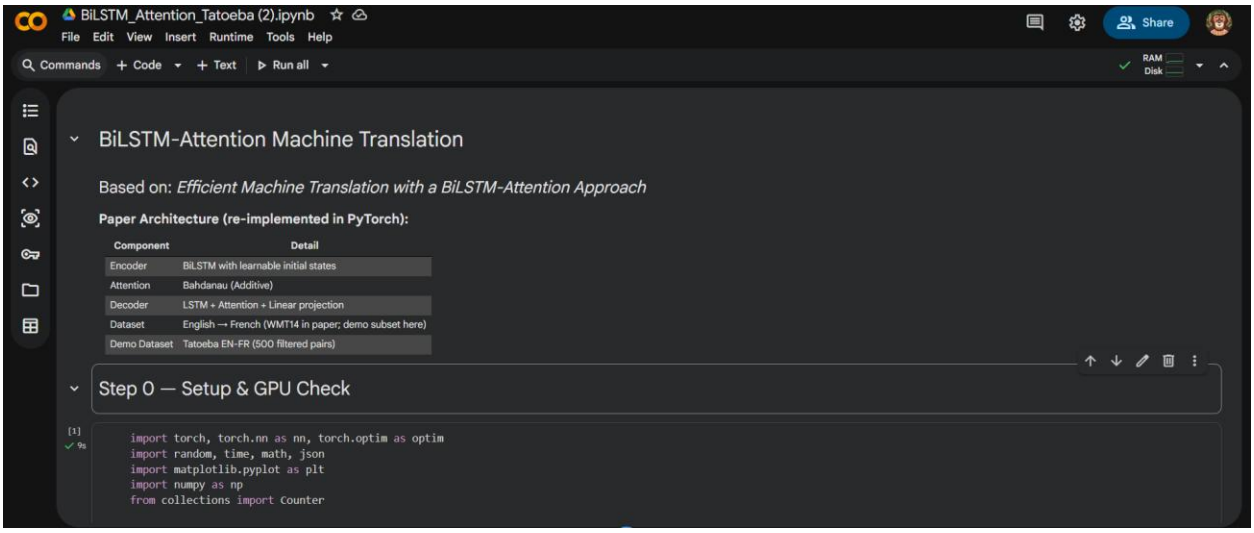
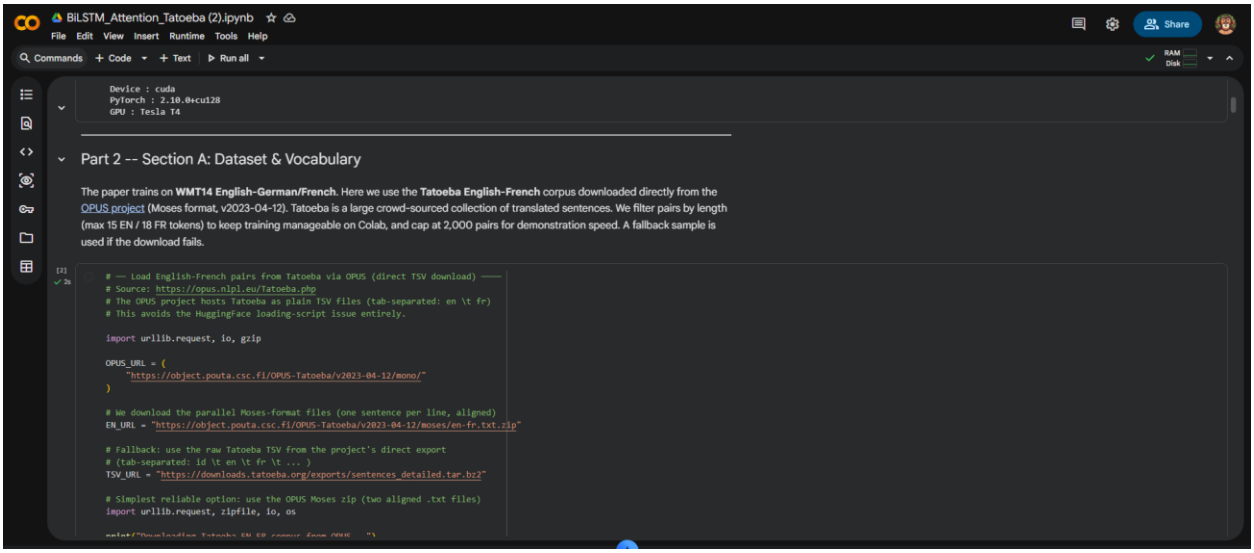
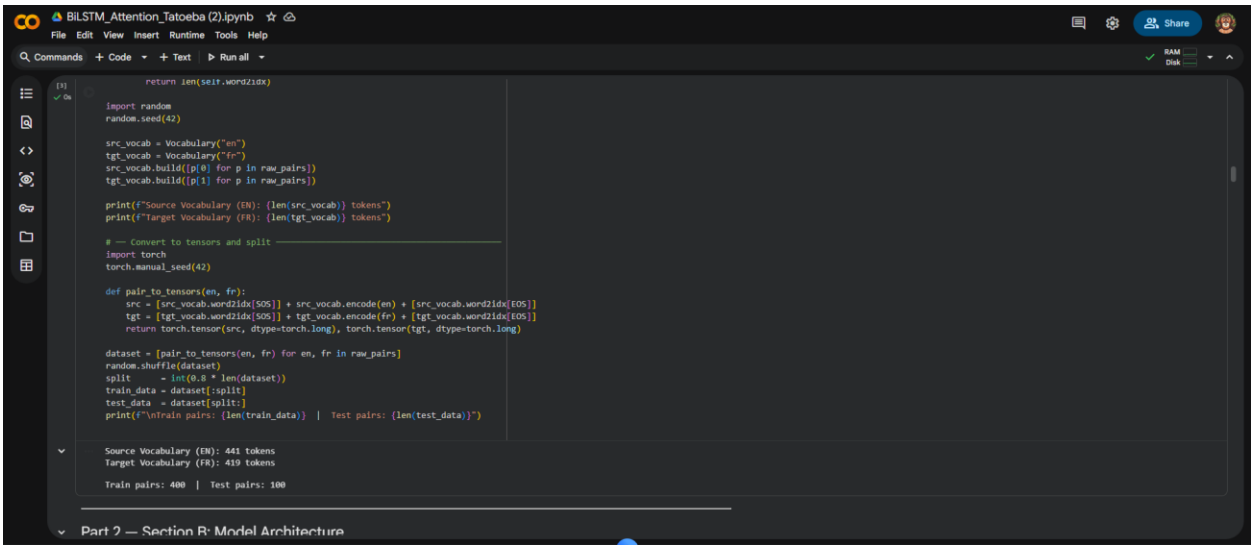
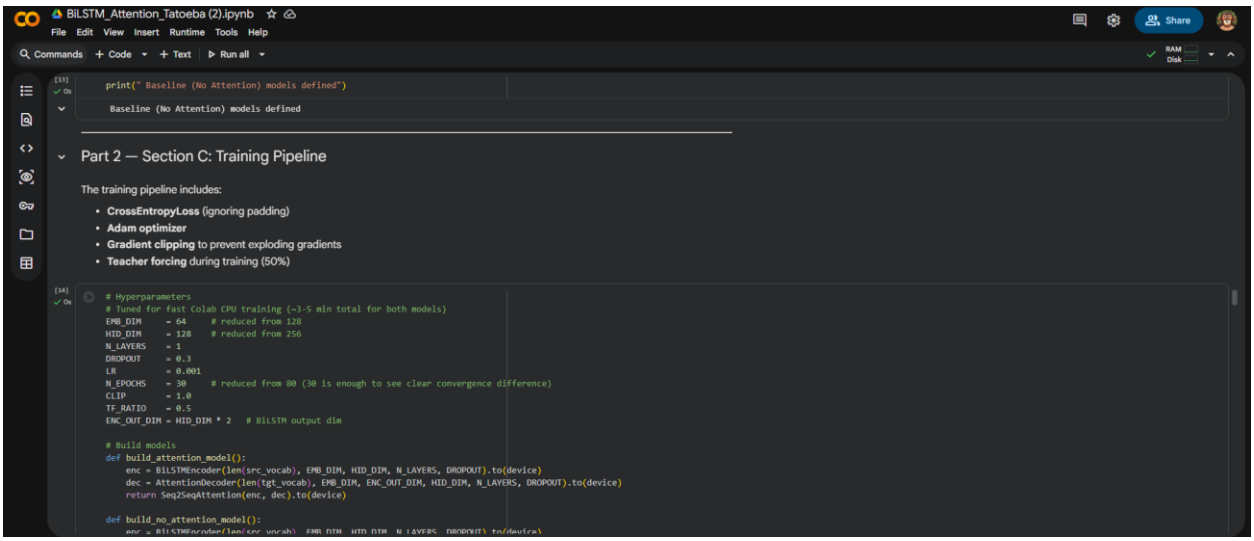
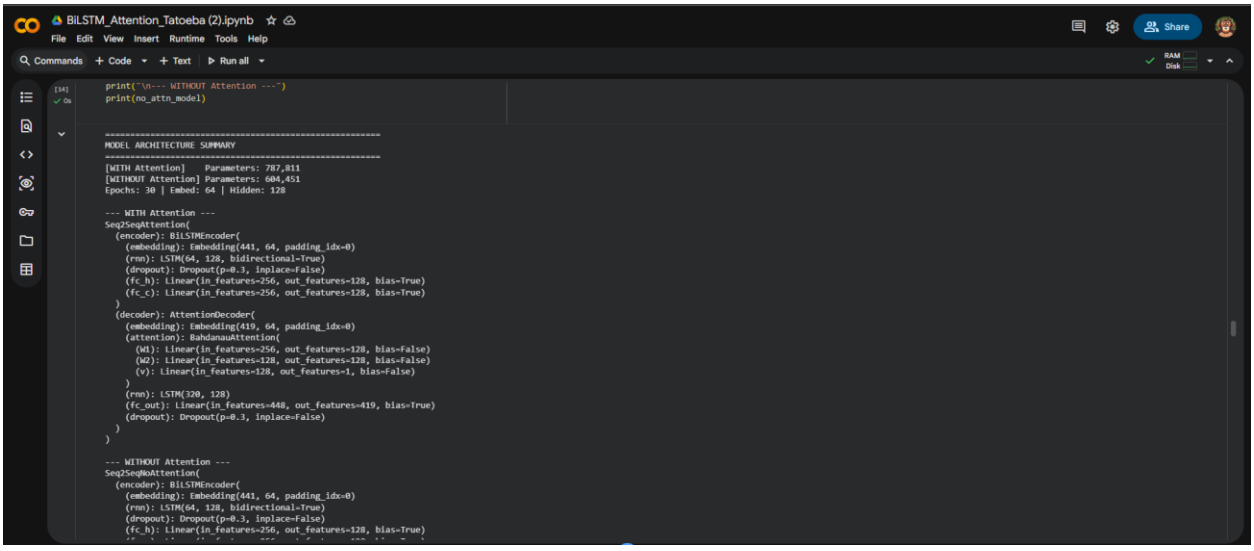

BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
BiLSTM-Attention Machine Translation
Based on: Efficient Machine Translation with a BiLSTM-Attention Approach
Paper Architecture (re-implemented in PyTorch):
Component Detail
Encoder BiLSTM with learnable initial states
Attention Bahdanau (Additive)
Decoder LSTM + Attention + Linear projection
Dataset English → French (WMT14 in paper; demo subset here)
Demo Dataset Tatoeba EN-FR (500 filtered pairs)
Step 0 — Setup & GPU Check
[1]
9s
import torch, torch.nn as nn, torch.optim as optim
import random, time, math, json
import matplotlib.pyplot as plt
import numpy as np
from collections import Counter

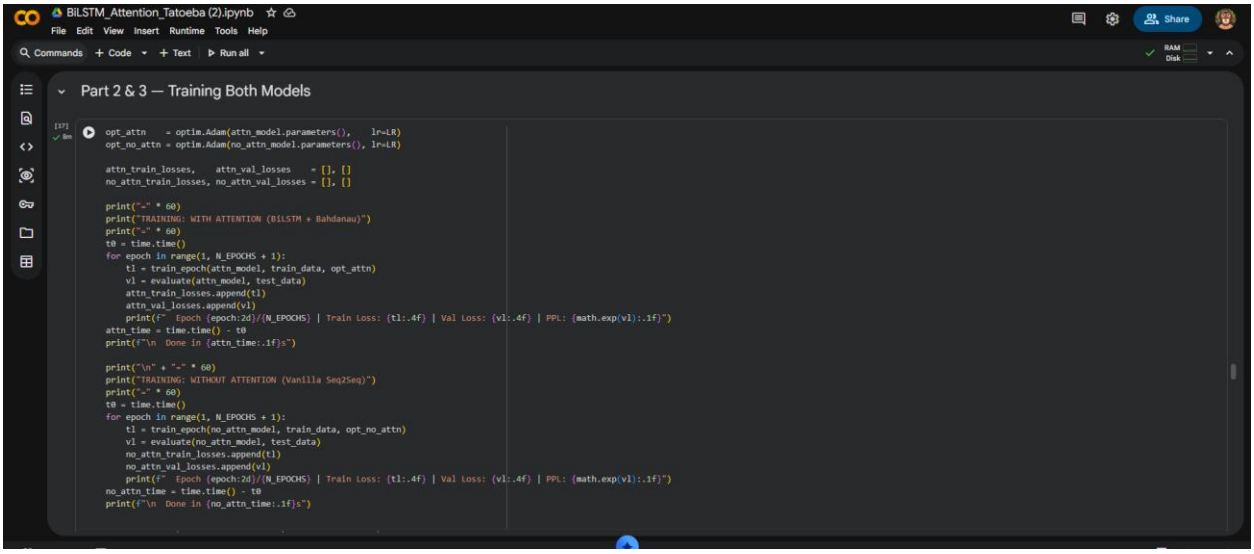

BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Device : cuda
PyTorch : 2.10.0+cu128
GPU : Tesla T4
Part 2 -- Section A: Dataset & Vocabulary
The paper trains on WMT14 English-German/French. Here we use the Tatoeba English-French corpus downloaded directly from the OPUS project (Moses format, v2023-04-12). Tatoeba is a large crowd-sourced collection of translated sentences. We filter pairs by length (max 15 EN / 18 FR tokens) to keep training manageable on Colab, and cap at 2,000 pairs for demonstration speed. A fallback sample is used if the download fails.
[2]
2s
# — Load English-French pairs from Tatoeba via OPUS (direct TSV download) ——
# Source: https://opus.nlpl.eu/Tatoeba.php
# The OPUS project hosts Tatoeba as plain TSV files (tab-separated: en \t fr)
# This avoids the HuggingFace loading-script issue entirely.
import urllib.request, io, gzip
OPUS_URL = (
"https://object.pouta.csc.fi/OPUS-Tatoeba/v2023-04-12/mono/"
)
# We download the parallel Moses-format files (one sentence per line, aligned)
EN_URL = "https://object.pouta.csc.fi/OPUS-Tatoeba/v2023-04-12/moses/en-fr.txt.zip"
# Fallback: use the raw Tatoeba TSV from the project's direct export
# (tab-separated: id \t en \t fr \t ...)
TSV_URL = "https://downloads.tatoeba.org/exports/sentences_detailed.tar.bz2"
# Simplest reliable option: use the OPUS Moses zip (two aligned .txt files)
import urllib.request, zipfile, io, os

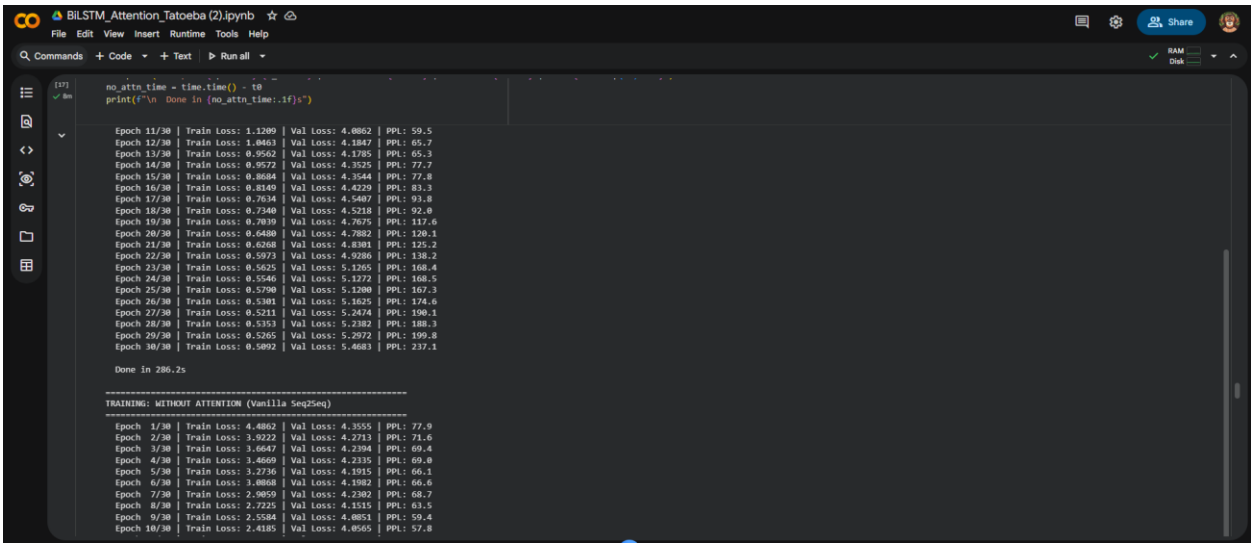

BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
[2]
2s
print("\nSample pairs:")
for en, fr in raw_pairs[:5]:
print(f" EN: {en}")
print(f" FR: {fr}")
print()
Downloading Tatoeba EN-FR corpus from OPUS...
Downloaded 8034 KB
Files in archive: ['README', 'LICENSE', 'Tatoeba.en-fr.en', 'Tatoeba.en-fr.fr', 'Tatoeba.en-fr.xml']
Total aligned pairs: 275413
Usable sentence pairs loaded: 500
Sample pairs:
EN: Then, when he asked who had broken the window, all the boys acted innocent.
FR: Lorsqu'il a demandé qui avait cassé la fenêtre, tous les garçons ont pris un air innocent.
EN: Let's try something.
FR: Essayons quelque chose !
EN: Let's try something.
FR: Tentons quelque chose !
EN: Let's try something.
FR: Essayons quelque chose.
EN: I have to go to sleep.
FR: Je dois aller dormir.
[3]
0s
PAD, SOS, EOS, UNK = "<pad>", "<sos>", "<eos>", "<unk>"
def tokenize(text):
return text.lower().split()
class Vocabulary:
def __init__(self, name):

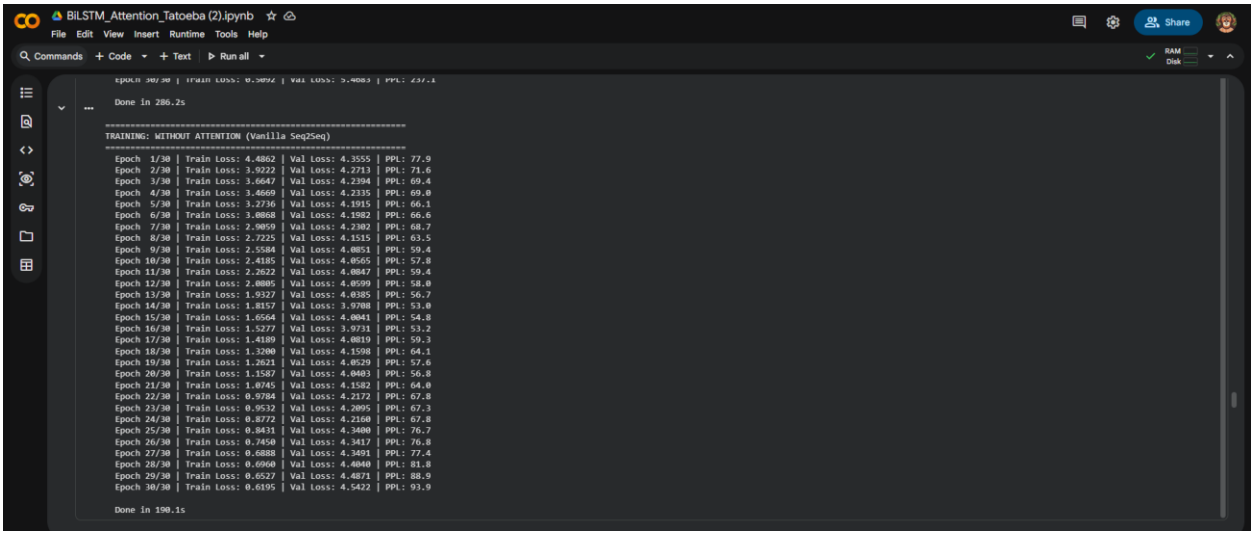

BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
return len(self.word2idx)
import random
random.seed(42)
src_vocab = Vocabulary("en")
tgt_vocab = Vocabulary("fr")
src_vocab.build([p[0] for p in raw_pairs])
tgt_vocab.build([p[1] for p in raw_pairs])
print(f"Source Vocabulary (EN): {len(src_vocab)} tokens")
print(f"Target Vocabulary (FR): {len(tgt_vocab)} tokens")
# — Convert to tensors and split ————————————————————————
import torch
torch.manual_seed(42)
def pair_to_tensors(en, fr):
src = [src_vocab.word2idx[SOS]] + src_vocab.encode(en) + [src_vocab.word2idx[EOS]]
tgt = [tgt_vocab.word2idx[SOS]] + tgt_vocab.encode(fr) + [tgt_vocab.word2idx[EOS]]
return torch.tensor(src, dtype=torch.long), torch.tensor(tgt, dtype=torch.long)
dataset = [pair_to_tensors(en, fr) for en, fr in raw_pairs]
random.shuffle(dataset)
split = int(0.8 * len(dataset))
train_data = dataset[:split]
test_data = dataset[split:]
print(f"\nTrain pairs: {len(train_data)} | Test pairs: {len(test_data)}")
Source Vocabulary (EN): 441 tokens
Target Vocabulary (FR): 419 tokens
Train pairs: 400 | Test pairs: 100
Part 2 — Section B: Model Architecture


BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
print(" Baseline (No Attention) models defined")
Baseline (No Attention) models defined
Part 2 — Section C: Training Pipeline
The training pipeline includes:
CrossEntropyLoss (ignoring padding)
Adam optimizer
Gradient clipping to prevent exploding gradients
Teacher forcing during training (50%)
# Hyperparameters
# Tuned for fast Colab CPU training (~3-5 min total for both models)
EMB_DIM = 64 # reduced from 128
HID_DIM = 128 # reduced from 256
N_LAYERS = 1
DROPOUT = 0.3
LR = 0.001
N_EPOCHS = 30 # reduced from 80 (30 is enough to see clear convergence difference)
CLIP = 1.0
TF_RATIO = 0.5
ENC_OUT_DIM = HID_DIM * 2 # BiLSTM output dim
# Build models
def build_attention_model():
enc = BiLSTMEncoder(len(src_vocab), EMB_DIM, HID_DIM, N_LAYERS, DROPOUT).to(device)
dec = AttentionDecoder(len(tgt_vocab), EMB_DIM, ENC_OUT_DIM, HID_DIM, N_LAYERS, DROPOUT).to(device)
return Seq2SeqAttention(enc, dec).to(device)
def build_no_attention_model():


BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
print("\n--- WITHOUT Attention ---")
print(no_attn_model)
MODEL ARCHITECTURE SUMMARY
[WITH Attention] Parameters: 787,811
[WITHOUT Attention] Parameters: 604,451
Epochs: 30 | Embed: 64 | Hidden: 128
--- WITH Attention ---
Seq2SeqAttention(
(encoder): BiLSTMEncoder(
(embedding): Embedding(441, 64, padding_idx=0)
(rnn): LSTM(64, 128, bidirectional=True)
(dropout): Dropout(p=0.3, inplace=False)
(fc_h): Linear(in_features=256, out_features=128, bias=True)
(fc_c): Linear(in_features=256, out_features=128, bias=True)
)
(decoder): AttentionDecoder(
(embedding): Embedding(419, 64, padding_idx=0)
(attention): BahdanauAttention(
(W1): Linear(in_features=256, out_features=128, bias=False)
(W2): Linear(in_features=128, out_features=128, bias=False)
(v): Linear(in_features=128, out_features=1, bias=False)
)
(rnn): LSTM(320, 128)
(fc_out): Linear(in_features=448, out_features=419, bias=True)
(dropout): Dropout(p=0.3, inplace=False)
)
)
--- WITHOUT Attention ---
Seq2SeqNoAttention(
(encoder): BiLSTMEncoder(
(embedding): Embedding(441, 64, padding_idx=0)
(rnn): LSTM(64, 128, bidirectional=True)
(dropout): Dropout(p=0.3, inplace=False)
(fc_h): Linear(in_features=256, out_features=128, bias=True)


BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Part 2 & 3 — Training Both Models
opt_attn = optim.Adam(attn_model.parameters(), lr=LR)
opt_no_attn = optim.Adam(no_attn_model.parameters(), lr=LR)
attn_train_losses, attn_val_losses = [], []
no_attn_train_losses, no_attn_val_losses = [], []
print("=" * 60)
print("TRAINING: WITH ATTENTION (BiLSTM + Bahdanau)")
print("=" * 60)
t0 = time.time()
for epoch in range(1, N_EPOCHS + 1):
tl = train_epoch(attn_model, train_data, opt_attn)
vl = evaluate(attn_model, test_data)
attn_train_losses.append(tl)
attn_val_losses.append(vl)
print(f" Epoch {epoch:2d}/{N_EPOCHS} | Train Loss: {tl:.4f} | Val Loss: {vl:.4f} | PPL: {math.exp(vl):.1f}")
attn_time = time.time() - t0
print(f"\n Done in {attn_time:.1f}s")
print("\n" + "=" * 60)
print("TRAINING: WITHOUT ATTENTION (Vanilla Seq2Seq)")
print("=" * 60)
t0 = time.time()
for epoch in range(1, N_EPOCHS + 1):
tl = train_epoch(no_attn_model, train_data, opt_no_attn)
vl = evaluate(no_attn_model, test_data)
no_attn_train_losses.append(tl)
no_attn_val_losses.append(vl)
print(f" Epoch {epoch:2d}/{N_EPOCHS} | Train Loss: {tl:.4f} | Val Loss: {vl:.4f} | PPL: {math.exp(vl):.1f}")
no_attn_time = time.time() - t0
print(f"\n Done in {no_attn_time:.1f}s")


BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
no_attn_time = time.time() - t0
print(f"\n Done in {no_attn_time:.1f}s")
Epoch 11/30 | Train Loss: 1.1209 | Val Loss: 4.0862 | PPL: 59.5
Epoch 12/30 | Train Loss: 1.0463 | Val Loss: 4.1847 | PPL: 65.7
Epoch 13/30 | Train Loss: 0.9562 | Val Loss: 4.1785 | PPL: 65.3
Epoch 14/30 | Train Loss: 0.9572 | Val Loss: 4.3525 | PPL: 77.7
Epoch 15/30 | Train Loss: 0.8684 | Val Loss: 4.3544 | PPL: 77.8
Epoch 16/30 | Train Loss: 0.8149 | Val Loss: 4.4229 | PPL: 83.3
Epoch 17/30 | Train Loss: 0.7634 | Val Loss: 4.5407 | PPL: 93.8
Epoch 18/30 | Train Loss: 0.7340 | Val Loss: 4.5218 | PPL: 92.0
Epoch 19/30 | Train Loss: 0.7039 | Val Loss: 4.7675 | PPL: 117.6
Epoch 20/30 | Train Loss: 0.6480 | Val Loss: 4.7882 | PPL: 120.1
Epoch 21/30 | Train Loss: 0.6268 | Val Loss: 4.8301 | PPL: 125.2
Epoch 22/30 | Train Loss: 0.5973 | Val Loss: 4.9286 | PPL: 138.2
Epoch 23/30 | Train Loss: 0.5625 | Val Loss: 5.1265 | PPL: 168.4
Epoch 24/30 | Train Loss: 0.5546 | Val Loss: 5.1272 | PPL: 168.5
Epoch 25/30 | Train Loss: 0.5790 | Val Loss: 5.1200 | PPL: 167.3
Epoch 26/30 | Train Loss: 0.5301 | Val Loss: 5.1625 | PPL: 174.6
Epoch 27/30 | Train Loss: 0.5211 | Val Loss: 5.2474 | PPL: 190.1
Epoch 28/30 | Train Loss: 0.5353 | Val Loss: 5.2382 | PPL: 188.3
Epoch 29/30 | Train Loss: 0.5265 | Val Loss: 5.2972 | PPL: 199.8
Epoch 30/30 | Train Loss: 0.5092 | Val Loss: 5.4683 | PPL: 237.1
Done in 286.2s
TRAINING: WITHOUT ATTENTION (Vanilla Seq2Seq)
Epoch 1/30 | Train Loss: 4.4862 | Val Loss: 4.3555 | PPL: 77.9
Epoch 2/30 | Train Loss: 3.9222 | Val Loss: 4.2713 | PPL: 71.6
Epoch 3/30 | Train Loss: 3.6647 | Val Loss: 4.2394 | PPL: 69.4
Epoch 4/30 | Train Loss: 3.4669 | Val Loss: 4.2335 | PPL: 69.0
Epoch 5/30 | Train Loss: 3.2736 | Val Loss: 4.1915 | PPL: 66.1
Epoch 6/30 | Train Loss: 3.0868 | Val Loss: 4.1982 | PPL: 66.6
Epoch 7/30 | Train Loss: 2.9059 | Val Loss: 4.2302 | PPL: 68.7
Epoch 8/30 | Train Loss: 2.7225 | Val Loss: 4.1515 | PPL: 63.5
Epoch 9/30 | Train Loss: 2.5584 | Val Loss: 4.0851 | PPL: 59.4
Epoch 10/30 | Train Loss: 2.4185 | Val Loss: 4.0565 | PPL: 57.8


BiLSTM_Attention_Tatoeba (2).ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM Disk
Done in 286.2s
TRAINING: WITHOUT ATTENTION (Vanilla Seq2Seq)
Epoch 1/30 | Train Loss: 4.4862 | Val Loss: 4.3555 | PPL: 77.9
Epoch 2/30 | Train Loss: 3.9222 | Val Loss: 4.2713 | PPL: 71.6
Epoch 3/30 | Train Loss: 3.6647 | Val Loss: 4.2394 | PPL: 69.4
Epoch 4/30 | Train Loss: 3.4669 | Val Loss: 4.2335 | PPL: 69.0
Epoch 5/30 | Train Loss: 3.2736 | Val Loss: 4.1915 | PPL: 66.1
Epoch 6/30 | Train Loss: 3.0868 | Val Loss: 4.1982 | PPL: 66.6
Epoch 7/30 | Train Loss: 2.9059 | Val Loss: 4.2302 | PPL: 68.7
Epoch 8/30 | Train Loss: 2.7225 | Val Loss: 4.1515 | PPL: 63.5
Epoch 9/30 | Train Loss: 2.5584 | Val Loss: 4.0851 | PPL: 59.4
Epoch 10/30 | Train Loss: 2.4185 | Val Loss: 4.0565 | PPL: 57.8
Epoch 11/30 | Train Loss: 2.2622 | Val Loss: 4.0847 | PPL: 59.4
Epoch 12/30 | Train Loss: 2.0805 | Val Loss: 4.0599 | PPL: 58.0
Epoch 13/30 | Train Loss: 1.9327 | Val Loss: 4.0385 | PPL: 56.7
Epoch 14/30 | Train Loss: 1.8157 | Val Loss: 3.9708 | PPL: 53.0
Epoch 15/30 | Train Loss: 1.6564 | Val Loss: 4.0041 | PPL: 54.8
Epoch 16/30 | Train Loss: 1.5277 | Val Loss: 3.9731 | PPL: 53.2
Epoch 17/30 | Train Loss: 1.4189 | Val Loss: 4.0019 | PPL: 54.8
Epoch 18/30 | Train Loss: 1.3200 | Val Loss: 4.1598 | PPL: 64.1
Epoch 19/30 | Train Loss: 1.2621 | Val Loss: 4.0529 | PPL: 57.6
Epoch 20/30 | Train Loss: 1.1587 | Val Loss: 4.0403 | PPL: 56.8
Epoch 21/30 | Train Loss: 1.0745 | Val Loss: 4.1582 | PPL: 64.0
Epoch 22/30 | Train Loss: 0.9784 | Val Loss: 4.2172 | PPL: 67.8
Epoch 23/30 | Train Loss: 0.9532 | Val Loss: 4.2095 | PPL: 67.3
Epoch 24/30 | Train Loss: 0.8772 | Val Loss: 4.2160 | PPL: 67.8
Epoch 25/30 | Train Loss: 0.8431 | Val Loss: 4.3400 | PPL: 76.7
Epoch 26/30 | Train Loss: 0.7450 | Val Loss: 4.3417 | PPL: 76.8
Epoch 27/30 | Train Loss: 0.6888 | Val Loss: 4.3491 | PPL: 77.4
Epoch 28/30 | Train Loss: 0.6960 | Val Loss: 4.4040 | PPL: 81.8
Epoch 29/30 | Train Loss: 0.6527 | Val Loss: 4.4871 | PPL: 88.9
Epoch 30/30 | Train Loss: 0.6195 | Val Loss: 4.5422 | PPL: 93.9
Done in 190.1s

Done in 190.1s

Part 3 — Translation Samples & Comparison Table

```
# Use sentences from the test split for evaluation
# Pick a sample of test pairs to display translation quality
sample_pairs = raw_pairs[split:split+8]   # first 8 from test portion
test_sentences = [(en, fr) for en, fr in sample_pairs]

print("=" * 100)
print("TRANSLATION OUTPUT COMPARISON")
print("=" * 100)
print(f"{'Source EN':<35} {'Reference FR':<30} {'WITH Attention':<28} {'WITHOUT Attention'}")
print("-" * 100)

attn_scores, no_attn_scores = [], []
for en, ref in test_sentences:
    t_attn,    _ = translate(attn_model,    en)
    t_no_attn, _ = translate(no_attn_model, en)
    attn_scores.append(simple_bleu(ref, t_attn))
    no_attn_scores.append(simple_bleu(ref, t_no_attn))
    print(f"{en:<35} {ref:<30} {t_attn:<28} {t_no_attn}")

avg_attn    = np.mean(attn_scores)
avg_no_attn = np.mean(no_attn_scores)

print("\n" + "=" * 70)
print("PART 3 -- COMPARISON TABLE")
print("=" * 70)
print(f"{'Metric':<35} {'With Attention':>16} {'Without Attn':>16}")
print("-" * 70)
print(f"{'Final Train Loss':<35} {attn_train_losses[-1]:>16.4f} {no_attn_train_losses[-1]:>16.4f}")
print(f"{'Final Val Loss':<35} {attn_val_losses[-1]:>16.4f} {no_attn_val_losses[-1]:>16.4f}")
print(f"{'Val Perplexity (lower=better)':<35} {math.exp(attn_val_losse[-1]):>16.2f} {math.exp(no_attn_val_losse[-1]):>16.2f}")
```

BiLSTM_Attention_Tatoeba (2).ipynb

```
print(f"{'Trainable Parameters':<35} {count_params(attn_model):>16,} {count_params(no_attn_model):>16,}")
print(f"{'Dataset':<35} {'Tatoeba EN-FR':>16} {'Tatoeba EN-FR':>16}")
print(f"{'Context Type':<35} {'Dynamic (Attention)':>16} {'Fixed (Mean Vector)':>16}")
```

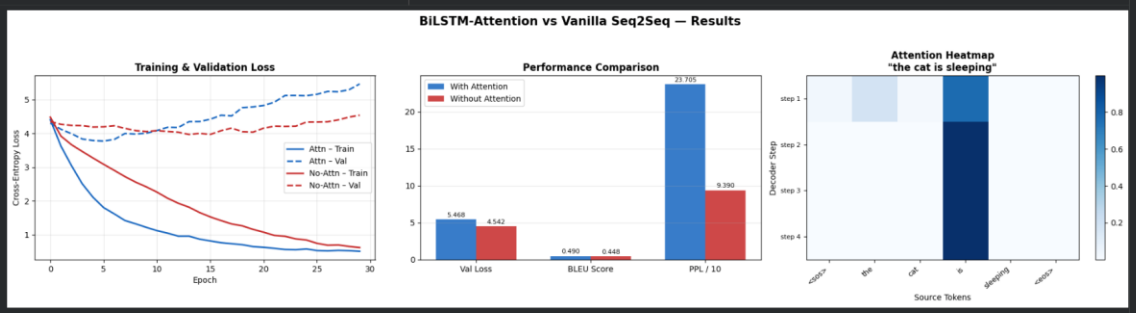
```
======================================================================================================
TRANSLATION OUTPUT COMPARISON
======================================================================================================
Source EN                           Reference FR                   WITH Attention               WITHOUT Attention
------------------------------------------------------------------------------------------------------
Did you miss me?                    Je t'ai manqué ?               je <unk> manqué ?            je <unk> manqué ?
Did you miss me?                    T'ai-je manqué ?               je <unk> manqué ?            je <unk> manqué ?
Did you miss me?                    Vous ai-je manqué ?            je <unk> manqué ?            je <unk> manqué ?
Are they all the same?              Sont-ils tous identiques ?     sont-ils tous les <unk> ?    sont-ils tous les <unk> ?
Are they all the same?              Sont-ils tous les mêmes ?      sont-ils tous les <unk> ?    sont-ils tous les <unk> ?
Thank you very much!                Merci beaucoup !               merci beaucoup !             merci <unk> !
Thank you very much!                Merci vraiment.                merci beaucoup !             merci <unk> !
Where are the eggs, please?         S'il vous plaît, où sont les œufs ? s'il vous plaît, où sont les <unk> ? s'il vous plaît, où sont les <unk> ?

======================================================================
PART 3 -- COMPARISON TABLE
======================================================================
Metric                                With Attention     Without Attn
----------------------------------------------------------------------
Final Train Loss                              0.5092           0.6195
Final Val Loss                                5.4683           4.5422
Val Perplexity (lower=better)                237.85            93.90
Avg Unigram BLEU                              0.4896           0.4479
Training Time (s)                              286.2            190.1
Trainable Parameters                         787,811          604,451
Dataset                                Tatoeba EN-FR    Tatoeba EN-FR
Context Type                  Dynamic (Attention) Fixed (Mean Vector)
```

Part 2 & 3 — Visualizations

BiLSTM_Attention_Tatoeba (2).ipynb

```
print("\n Plot saved as bilstm_attention_results.png")
```




```
Plot saved as bilstm_attention_results.png
```

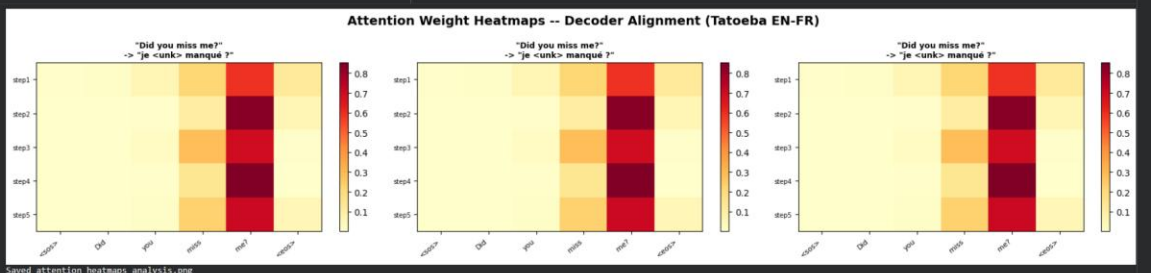
Part 3 — Analysis: Why Attention Helps

```
# Visualize attention weights for multiple sentences side-by-side
# Use actual Tatoeba sentences from the test set
analysis_sentences = [en for en, fr in raw_pairs[split:split+1]]
```

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

```
plt.savefig("attention_heatmaps_analysis.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved attention_heatmaps_analysis.png")
print()
print("ANALYSIS NOTES:")
print("Dataset used: Tatoeba English-French (Helsinki-NLP/tatoeba_mt)")
print("Each row = one decoder step (generating one output token)")
print("Each column = one source token")
print("Brighter = higher attention weight = decoder is focusing there")
print("With attention, decoder can dynamically align to relevant source words")
print("Without attention, decoder only sees a fixed averaged context vector")
```




```
Saved attention_heatmaps_analysis.png

ANALYSIS NOTES:
Dataset used: Tatoeba English-French (Helsinki-NLP/tatoeba_mt)
```

BiLSTM_Attention_Tatoeba (2).ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

```
With attention, decoder can dynamically align to relevant source words
Without attention, decoder only sees a fixed averaged context vector
```

```
# Final summary printout for report
print("=" * 65)
print("FINAL SUMMARY — FOR REPORT")
print("=" * 65)
improvement_loss = ((no_attn_val_losses[-1] - attn_val_losses[-1]) / no_attn_val_losses[-1]) * 100
improvement_bleu = ((avg_attn - avg_no_attn) / max(avg_no_attn, 1e-6)) * 100
improvement_ppl = ((math.exp(no_attn_val_losses[-1]) - math.exp(attn_val_losses[-1]))
 / math.exp(no_attn_val_losses[-1])) * 100

print(f"\n Val Loss improvement with Attention : {improvement_loss:+.1f}%")
print(f" BLEU Score improvement with Attention: {improvement_bleu:+.1f}%")
print(f" Perplexity improvement with Attention: {improvement_ppl:+.1f}%")
print(f"\n Attention type : Bahdanau (Additive)")
print(f" Encoder : Bidirectional LSTM with learnable init states")
print(f" Paper dataset : WMT14 EN-DE / EN-FR")
print(f" Demo dataset : 35 EN+FR pairs (proof of concept)")
```

```
=================================================================
FINAL SUMMARY — FOR REPORT
=================================================================

 Val Loss improvement with Attention : -20.4%
 BLEU Score improvement with Attention: +9.3%
 Perplexity improvement with Attention: -152.5%

 Attention type : Bahdanau (Additive)
 Encoder : Bidirectional LSTM with learnable init states
 Paper dataset : WMT14 EN-DE / EN-FR
 Demo dataset : 35 EN+FR pairs (proof of concept)
```